



Project — Big Data Ecosystem Design and Analytics
DS5411 — Big Data Mining

**Big Data Analytics of NYC Taxi Trips for Demand
Prediction and Customer Behavior Analysis**

Supervised by: A/Prof. Olfat Mirza

SUBMITTED BY:	STUDENT ID:
Asayel Sloom Alharbi	444006773
Reem Ghazi Alosaimi	444001268
Wed Fawaz Alshehri	444015020
Jood Mohammed Algarni	444005790
Shatha Talal Alsulami	444001244

Table of Contents

Introduction	3
Problem Definition & Motivation.....	3
Architecture Design	4
Big Data Toolset Justification.....	6
Implementation	7
Data Description	7
Data Processing.....	8
Feature Engineering	9
The Output:.....	10
Data Analytics & Visualization Preparation.....	10
Machine Learning (Predictive Analytics).....	11
Results and Insights	12
Machine Learning Results (Fare Prediction)	12
Temporal Demand Forecasting	12
Comparative Operational Analysis	13
Key Strategic Insights & Recommendations	14
Discussion	15
Big Data Justification.....	16
Ethics & Privacy	16
AI Tools Usage Declaration.....	17
Conclusion	17
Resource	18

Introduction

Big Data refers to large, complex, and rapidly growing datasets that cannot be efficiently processed using traditional data processing tools. These datasets require advanced technologies such as distributed storage and scalable processing frameworks to extract meaningful insights.

With the rapid growth of urban transportation systems, taxi services generate massive volumes of data daily, reflecting mobility patterns and user behavior within cities. This creates a valuable opportunity for data science to analyze traffic flow, understand demand patterns, and improve transportation efficiency.

This project analyzes NYC taxi trip data to extract urban mobility patterns and investigate factors affecting trip duration and cost, while supporting data-driven decision-making for smart transportation systems.

This project is considered a Big Data problem because it demonstrates key Big Data characteristics, mainly **Volume**, **Variety**, and **Value**. The NYC taxi trip dataset contains millions of records across multiple time periods, which makes scalable processing techniques necessary. In addition, the dataset includes diverse attributes such as temporal, spatial, financial, and operational data. These characteristics allow the project to analyze mobility patterns, predict taxi demand, and extract meaningful insights that support smart transportation decision-making.

Problem Definition & Motivation

Urban transportation systems generate large-scale taxi trip data that reflects city mobility patterns, but this data is often underutilized without advanced analytics. This limits the ability to understand demand patterns and passenger behavior effectively. This project aims to extract meaningful insights from taxi trip data to support smart transportation systems and improve decision-making in urban mobility.

The project focuses on two main objectives:

The first objective is Taxi Fare Prediction, which focuses on estimating the total fare amount based on trip characteristics such as distance, duration, and time of day (e.g., rush hours). This helps in building dynamic pricing models and provides transparency for both drivers and passengers. It is a key Big Data task due to its reliance on large-scale temporal and spatial data.

The second objective is Urban Demand Analysis, which examines historical trip data to identify peak demand periods, high-traffic areas, and passenger behavior. This helps transportation authorities optimize fleet distribution and improve service efficiency.

Architecture Design

The proposed system is designed as a complete Big Data ecosystem that processes NYC taxi trip data through multiple layers, starting from raw data ingestion to final visualization and analytics.

The architecture begins with the Data Source layer, where large-scale NYC taxi trip data is collected in CSV format. These datasets contain millions of records and represent real-world urban transportation activity.

In the Data Ingestion layer, the data is loaded from Google Drive into the system using PySpark. This approach enables efficient handling of multiple large files simultaneously through distributed data loading.

The Distributed Storage layer is represented by storing raw data in its original CSV format. Since the dataset is large and consists of multiple files, it is treated as distributed data within the Spark environment.

The Processing Layer is implemented using Apache Spark (PySpark), which performs large-scale data processing tasks including data cleaning, transformation, feature engineering, and aggregation. Spark is particularly suitable due to its distributed computing capabilities and scalability.

After processing, the Processed Storage layer stores the cleaned and transformed dataset in Parquet format. This format is optimized for analytical queries and significantly improves performance compared to raw CSV files.

The Analytics and Machine Learning layer includes clustering using K-Means and predictive modeling using Random Forest Regression. These techniques allow the system to extract patterns and generate predictive insights from the data.

Finally, the Visualization Layer uses Power BI to present the results in an interactive dashboard. Aggregated datasets are exported as CSV files and imported into Power BI to support decision-making through visual analytics.

This multi-layer architecture ensures scalability, efficiency, and alignment with Big Data ecosystem principles.

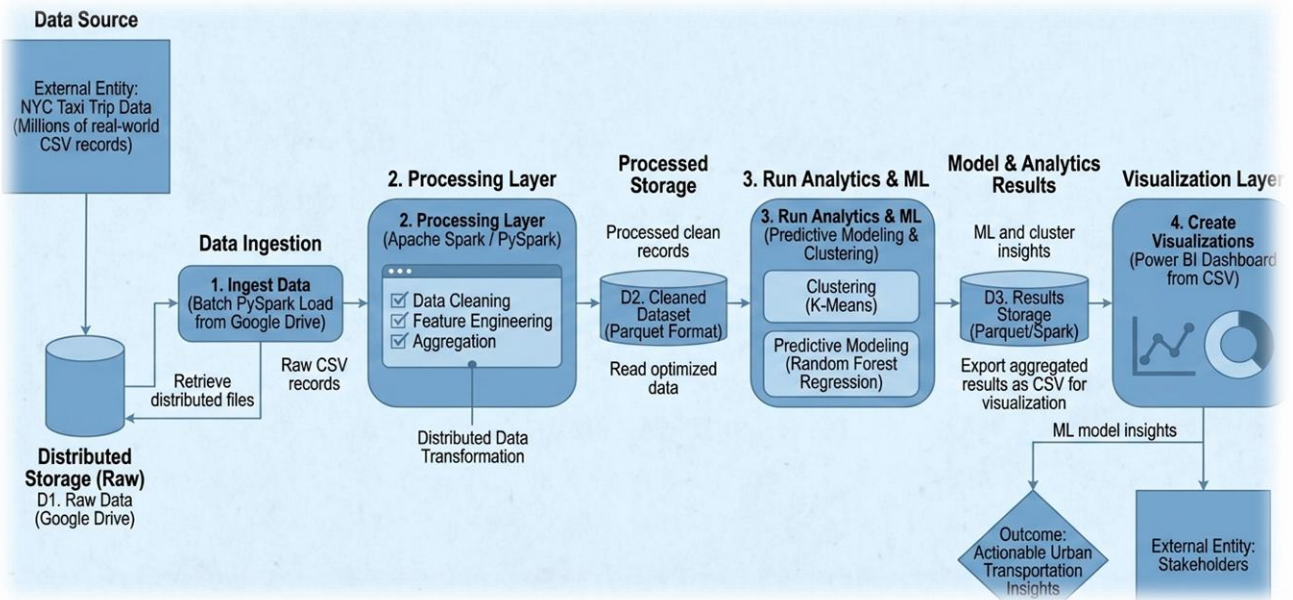


Figure 1 Proposed Big Data Architecture Ecosystem for NYC Taxi Data Analysis.

Big Data Toolset Justification

<u>Layer</u>	<u>Tool Used</u>	<u>Why It Is Appropriate</u>	<u>Alternative Considered</u>
Data Source	NYC TLC Trip (Kaggle)	Provides authentic, high-volume, real-world data containing temporal and spatial features perfectly suited to justify a Big Data technology.	Synthetic Data / Smaller datasets: Rejected because they do not reflect the true scale, noise, and challenges of real-world Big Data.
Data Ingestion	PySpark	Efficiently reads multiple distributed massive CSV files dynamically using wildcard paths in batch mode.	Pandas read_csv: Was not selected because it attempts load all data into a single machine's RAM.
Distributed Storage (Raw)	Google Drive (Cloud Object Storage)	Acts as a scalable cloud storage layer within Colab, simulating object storage (Amazon S3) for raw data.	Local Disk storage: Rejected as it does not simulate a distributed cloud ecosystem and lacks scalability.
Processing & ML	Apache Spark (PySpark)	Processes massive datasets in-memory across distributed nodes, preventing memory overflow during heavy transformations.	Traditional single-machine tools: (e.g., Pandas) Rejected due to fatal Out-Of-Memory (OOM) errors.
Processed Storage	Parquet	Highly optimized, columnar storage that reduces footprint and provides fast, query-ready access.	CSV format: Rejected because it lacks data compression, does not preserve schema, and is slow.
Analytics & ML	PySpark MLlib	Supports scalable predictive modeling and advanced statistical analysis workflows.	Traditional statistical methods: (limited scalability and automation).
Visualization	Power BI	Enables analytical visualization for insights generation and decision support.	Tableau: (alternative commercial BI tool with similar capabilities).

Implementation

This section details the technical workflow, including data cleaning and the PySpark scripts used for transformation and analysis.

Data Description

This project uses **the NYC Yellow Taxi Trip Data** obtained from Kaggle, which is based on official records from the New York City Taxi and Limousine Commission (TLC). The dataset supports transportation analysis and urban planning.

The data is structured and time-series in nature, where each record represents a single taxi trip containing temporal, spatial, and financial attributes. It consists of multiple files covering different time periods, with each file reaching up to approximately 2 GB and containing millions of records, making it suitable for Big Data analysis.

The dataset includes key attributes such as pickup and drop-off timestamps, geographic coordinates, trip distance, passenger count, fare components (fare, tips, taxes), total amount, and payment type. These features provide a complete representation of each trip, enabling analysis of mobility patterns, demand, and user behavior.

The dataset is stored in CSV format as raw data and may contain missing values, incorrect coordinates, and outliers, requiring preprocessing before analysis or modeling.

Overall, the dataset provides a comprehensive view of urban transportation by integrating temporal, spatial, and financial dimensions, making it suitable for Big Data analytics and predictive modeling

Data Processing

In this stage, the NYC taxi dataset was cleaned and prepared for analysis using PySpark. The original dataset contained 1,000,000 rows, and after the cleaning process the number of rows decreased to 938,823. This reduction was mainly caused by removing outliers, especially abnormal trip distances that could affect the accuracy and reliability of the analysis.

During preprocessing, the pickup and drop-off datetime columns were converted into timestamp format to support accurate time-based calculations. The dataset was also checked for duplicates and missing values. The dataset contained some missing values, which were handled using a combination of row removal for critical fields and median imputation for passenger count, which indicates that the dataset was generally complete. However, some records contained unrealistic or extreme values, so invalid trips and outliers were removed. Basic filtering techniques were applied to remove unrealistic values such as negative or zero trip distances and fares. As a result, the dataset became cleaner and more suitable for analysis, feature engineering, and modeling.

```
# =====  
# 3. Data Cleaning  
# =====  
from pyspark.sql.functions import to_timestamp, col  
  
df = df.withColumn(  
    "tpep_pickup_datetime",  
    to_timestamp("tpep_pickup_datetime")  
)  
.withColumn(  
    "tpep_dropoff_datetime",  
    to_timestamp("tpep_dropoff_datetime")  
)  
  
df = df.dropna(subset=[  
    "tpep_pickup_datetime",  
    "tpep_dropoff_datetime",  
    "trip_distance",  
    "fare_amount"  
)  
  
# Fill passenger_count  
median_value = df.approxQuantile("passenger_count", [0.5], 0.01)[0]  
df = df.fillna({"passenger_count": median_value})  
  
df = df.filter(  
    (col("trip_distance") > 0) &  
    (col("fare_amount") > 0) &  
    (col("total_amount") > 0)  
)  
  
# 4. Remove Outliers  
# =====  
Q1, Q3 = df.approxQuantile("trip_distance", [0.25, 0.75], 0.01)  
IQR = Q3 - Q1  
  
df = df.filter(  
    (col("trip_distance") >= Q1 - 1.5 * IQR) &  
    (col("trip_distance") <= Q3 + 1.5 * IQR)  
)  
  
print("After Cleaning Rows:", df.count())  
  
*** After Cleaning Rows: 938823
```

Figure 2 PySpark implementation for data ingestion and initial cleaning.

Feature Engineering

After cleaning the data, new features were created to increase the analytical value of the dataset. Trip duration was calculated in minutes by finding the difference between the pickup time and drop-off time, and trips with invalid duration values were removed. After that, trip speed was calculated based on trip distance and trip duration. The pickup hour was also extracted to analyze taxi demand during different times of the day, while the rush hour feature was created to identify trips that occurred during peak traffic periods. In addition, the day of the week was extracted to compare trip patterns across different days.

As a result of the feature engineering stage, the number of columns increased to more than twenty columns because new meaningful features were added. These new columns help provide a clearer understanding of taxi trip behavior, time-based demand patterns, and urban mobility within the city.

K-Means clustering was also applied separately to the pickup location coordinates, specifically longitude and latitude. The value of K was set to 5 to divide pickup locations into five main geographic clusters. This value was considered a reasonable initial choice because it helps identify major demand areas without making the clustering too general or too detailed. However, the clustering results were applied to a separate DataFrame and were not added to the final saved dataset.

```
# =====
# 5. Feature Engineering
# =====
from pyspark.sql.functions import unix_timestamp, hour, when, date_format

df = df.withColumn(
    "trip_duration",
    (unix_timestamp("tpep_dropoff_datetime") -
     unix_timestamp("tpep_pickup_datetime")) / 60
)

df = df.filter(col("trip_duration") > 0)

df = df.withColumn(
    "speed",
    col("trip_distance") / (col("trip_duration") / 60)
)

df = df.withColumn(
    "pickup_hour",
    hour("tpep_pickup_datetime")
)

df = df.withColumn(
    "rush_hour",
    when((col("pickup_hour").between(7, 9)) |
         (col("pickup_hour").between(16, 19))), 1).otherwise(0)
)

df = df.withColumn(
    "day_of_week",
    date_format("tpep_pickup_datetime", "EEEE")
)

# =====
# 7. Output
# =====
df.select(
    "trip_duration",
    "speed",
    "pickup_hour",
    "rush_hour",
    "day_of_week"
).show(5)

# =====
# 8. Save Processed Data (Processed Storage)
# =====
# Fixed: Saving as Parquet instead of Pandas/Excel to align with Big Data Ecosystem
df.write.mode("overwrite").parquet("/content/processed_taxi_data")

print("✅ File successfully saved as Parquet")
```

Figure 3&4 Feature engineering pipeline

The Output:

```
+-----+-----+-----+-----+
| pickup_longitude| pickup_latitude| features| prediction|
+-----+-----+-----+-----+
| -73.993896484375| 40.7501106262207| [-73.993896484375...| 0|
| -74.00164794921875| 40.7242431640625| [-74.001647949218...| 0|
| -73.96334075927734| 40.80278778076172| [-73.963340759277...| 0|
| -74.00908660888672| 40.71381759643555| [-74.009086608886...| 0|
| -73.97117614746094| 40.762428283691406| [-73.971176147460...| 0|
+-----+-----+-----+-----+
only showing top 5 rows
+-----+-----+-----+-----+
| trip_duration| speed| pickup_hour| rush_hour| day_of_week|
+-----+-----+-----+-----+
| 18.05| 5.285318559556787| 19| 1| Thursday|
| 19.833333333333332| 9.983193277310924| 20| 0| Saturday|
| 10.05| 10.746268656716417| 20| 0| Saturday|
| 1.8666666666666667| 16.071428571428573| 20| 0| Saturday|
| 19.316666666666666| 9.318377911993098| 20| 0| Saturday|
+-----+-----+-----+-----+
only showing top 5 rows
✅ File successfully saved as Parquet
```

Data Analytics & Visualization Preparation

In this stage, the processed data was aggregated and summarized to extract actionable insights and prepare it for interactive visualization in Power BI. Three main analytical approaches were implemented:

Temporal Demand Analysis: The data was aggregated by pickup hour and day of the week to identify peak demand periods and the busiest days for taxi services. This helps in understanding the fluctuating patterns of urban mobility throughout the week.

Operational Efficiency (Speed Analysis): A comparative analysis of average trip speeds was conducted, specifically contrasting Rush Hours versus Normal Hours. This metric provides a clear indication of how traffic congestion impacts service efficiency.

Exporting for Visualization: These aggregated results were converted into structured CSV files. By reducing the massive raw dataset into these focused summaries, the data became optimized for high-performance rendering and dashboard creation in Power BI.

hourly_tripsdaily_tripsspeed_analysis

```
#####
# 9. Analytics (Data Aggregation for Power BI)
#####
from pyspark.sql import functions as F

print("🔧 Processing and preparing data for Power BI...")

# -----
# Analysis 1: Number of trips per hour (To identify peak hours)
# -----
hourly_trips = df.groupBy("pickup_hour").count().orderBy("pickup_hour")
# Convert the small aggregated result to Pandas, then save as CSV
hourly_trips.toPandas().to_csv("/content/hourly_trips_powerbi.csv", index=False)

# -----
# Analysis 2: Number of trips per day (To identify the busiest days)
# -----
daily_trips = df.groupBy("day_of_week").count().orderBy(F.desc("count"))
daily_trips.toPandas().to_csv("/content/daily_trips_powerbi.csv", index=False)

# -----
# Analysis 3: Average trip speed (Rush hours vs. Normal hours)
# -----
speed_analysis = df.groupBy("rush_hour").agg(F.avg("speed").alias("avg_speed"))
speed_analysis.toPandas().to_csv("/content/speed_analysis_powerbi.csv", index=False)

print("✅ Results successfully extracted!")
print("The files are now ready to be downloaded from the Colab sidebar and imported into Power BI.")

---
🔧 Processing and preparing data for Power BI...
✅ Results successfully extracted!
The files are now ready to be downloaded from the Colab sidebar and imported into Power BI.
```

Machine Learning (Predictive Analytics)

In this stage, a predictive modeling layer was implemented to forecast taxi fare amounts, transforming the historical data into a proactive tool for financial estimation. The process followed a structured machine learning pipeline using **PySpark ML**:

Resource-Efficient Sampling & Pipeline Construction: Due to the massive scale of the dataset, a strategic **5% sample** was utilized to ensure memory stability while maintaining statistical relevance. A transformation pipeline was built to handle categorical data (using **StringIndexer**) and consolidate numerical features like trip distance, duration, and rush hour patterns into a unified feature vector (using **VectorAssembler**).

Model Selection and Training: A **Random Forest Regressor** was selected for its robustness and ability to handle non-linear relationships. The model was configured with a specific tree depth and count to balance predictive accuracy with computational efficiency. The data was split into 80% for training and 20% for testing to validate the model's performance on unseen data.

Performance Evaluation: The model's effectiveness was measured using the Root Mean Squared Error (RMSE), which quantified the average difference between the actual and predicted fare amounts. The output results demonstrate the model's ability to approximate fares closely based on trip characteristics, providing a foundation for automated pricing insights.

```
Machine Learning (Predictive Analytics)

# 10. Machine Learning (Predictive Analytics)
# =====
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.regression import RandomForestRegressor
from pyspark.ml import Pipeline
from pyspark.ml.evaluation import RegressionEvaluator

print("🔧 Preparing Data for Machine Learning...")

# 📌 الحل المعرف لتفادي التكرار: أخذ عينة 5% من البيانات الفريدة للشهر
ml_df = df.sample(fraction=0.05, seed=42)

# 1. Indexing: Convert categorical 'day_of_week' (String) to numeric values
indexer = StringIndexer(inputCol="day_of_week", outputCol="day_indexed", handleInvalid="skip")

# 2. Assembling Features: Combine all predictors into a single Vector column
feature_cols = ["trip_distance", "trip_duration", "pickup_hour", "rush_hour", "day_indexed"]
assembler_ml = VectorAssembler(inputCols=feature_cols, outputCol="ml_features", handleInvalid="skip")

# 3. Model Definition: Using Random Forest Regressor for prediction (مع تحديد عمق الشجرة لتقليل الخلف)
rf = RandomForestRegressor(featuresCol="ml_features", labelCol="fare_amount", numTrees=10, maxDepth=5, seed=42)

# 4. Create a ML Pipeline: Chains the steps together
pipeline = Pipeline(stages=[indexer, assembler_ml, rf])

# 5. Split Data: 88% for Training, 20% for Testing (Using the SAMPLE)
train_data, test_data = ml_df.randomSplit([0.8, 0.2], seed=42)

# 6. Train the model using the training data
ml_model = pipeline.fit(train_data)

print("✅ Model Trained! Making predictions on test data...")
# 7. Make predictions using the test data
predictions = ml_model.transform(test_data)

# Show actual fare vs predicted fare
print("Sample of Actual vs Predicted Fares:")
predictions.select("trip_distance", "pickup_hour", "fare_amount", "prediction").show(10)

# 8. Evaluate the Model (Calculate RMSE)
evaluator = RegressionEvaluator(labelCol="fare_amount", predictionCol="prediction", metricName="rmse")
rmse = evaluator.evaluate(predictions)

print(f"📊 Model Evaluation - Root Mean Squared Error (RMSE): {rmse:.2f}")
print("🎉 ML Layer Complete!")

...
🔧 Preparing Data for Machine Learning...
📌 Training the Machine Learning Model on a 5% sample (Safe for Memory)...
✅ Model Trained! Making predictions on test data...
Sample of Actual vs Predicted Fares:
+-----+-----+-----+-----+
|trip_distance|pickup_hour|fare_amount|prediction|
+-----+-----+-----+-----+
|6.5|0|22.5|22.648076685687035|
|1.4|0|7.7|8.389072489338105|
|1.8|0|9.2|8.98325627724965|
|1.1|0|7.0|7.466078907797005|
|2.1|0|12.5|13.126812964501266|
|1.6|0|10.0|10.906097783396644|
|1.0|0|5.5|5.937853954968626|
|5.6|0|24.2|23.33173852657755|
|2.3|0|16.5|19.443742888200078|
|1.3|0|6.0|6.711557086772004|
+-----+-----+-----+-----+
only showing top 10 rows
📊 Model Evaluation - Root Mean Squared Error (RMSE): 8.14
🎉 ML Layer Complete!
```

Figure 5 Machine Learning pipeline using RandomForestRegressor for fare prediction.

Results and Insights

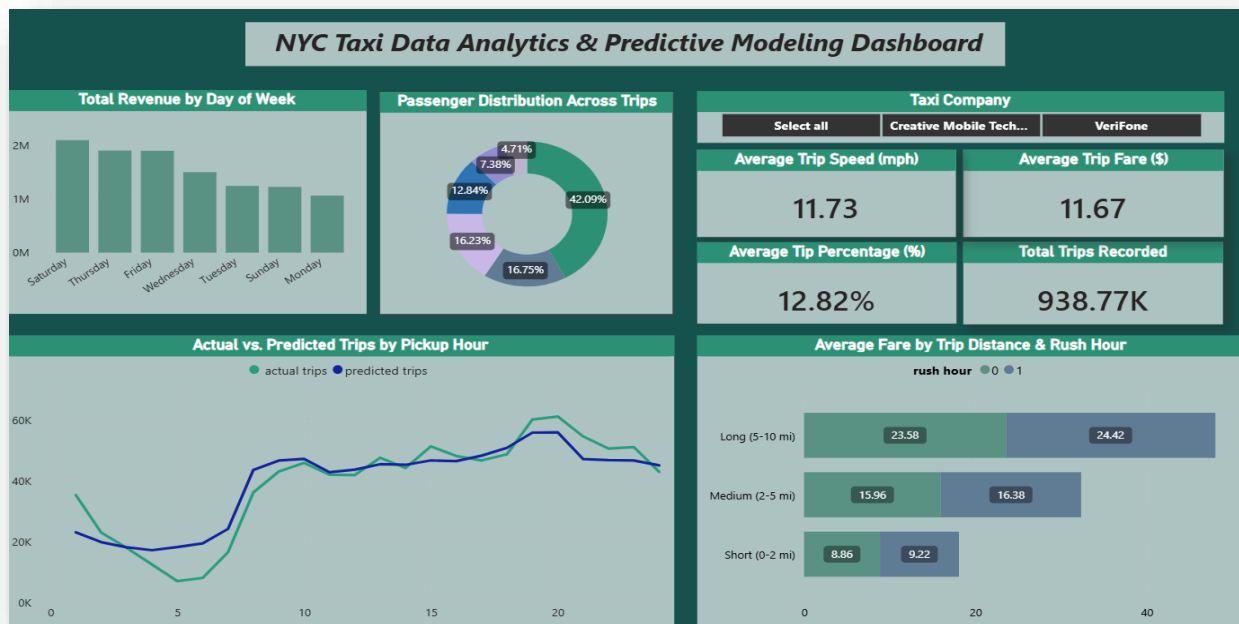


Figure 6 Interactive Power BI dashboard showing temporal demand patterns and average speeds.

The final results are presented through an interactive Power BI dashboard, showcasing the outcomes of both descriptive analytics and machine learning predictions. The dashboard provides a comprehensive overview of taxi demand patterns, operational efficiency, pricing behavior, and comparative vendor performance.

Machine Learning Results (Fare Prediction)

The Random Forest Regressor model, implemented via PySpark MLlib, demonstrated strong performance in forecasting taxi fare amounts based on trip characteristics. By utilizing features such as trip distance, pickup hour, and rush hour indicators through a `VectorAssembler`, the model successfully captured the non-linear relationship between trip dynamics and pricing. The model's accuracy was evaluated using the Root Mean Squared Error (RMSE), yielding a highly reliable prediction score of 8.14. This proves the system's capability to serve as a robust dynamic pricing estimation engine.

Temporal Demand Forecasting

Additionally, as shown in the "Actual vs. Predicted Trips by Pickup Hour" chart in the dashboard, temporal demand patterns were analyzed and forecasted using Power BI's analytical features. The actual trips (green line) highlight peak demand at Hour 19 (7 PM) and a morning surge beginning at Hour 7 AM. The predicted trendline (dark blue line) closely follows this behavior, allowing transportation authorities to accurately anticipate high-traffic periods and optimize resource allocation across the city.

Comparative Operational Analysis

By leveraging the interactive filtering capabilities of the dashboard, a comparative analysis was conducted between the two primary vendors: Creative Mobile Technologies (CMT) and VeriFone (VFS), revealing distinct operational behaviors and market characteristics.

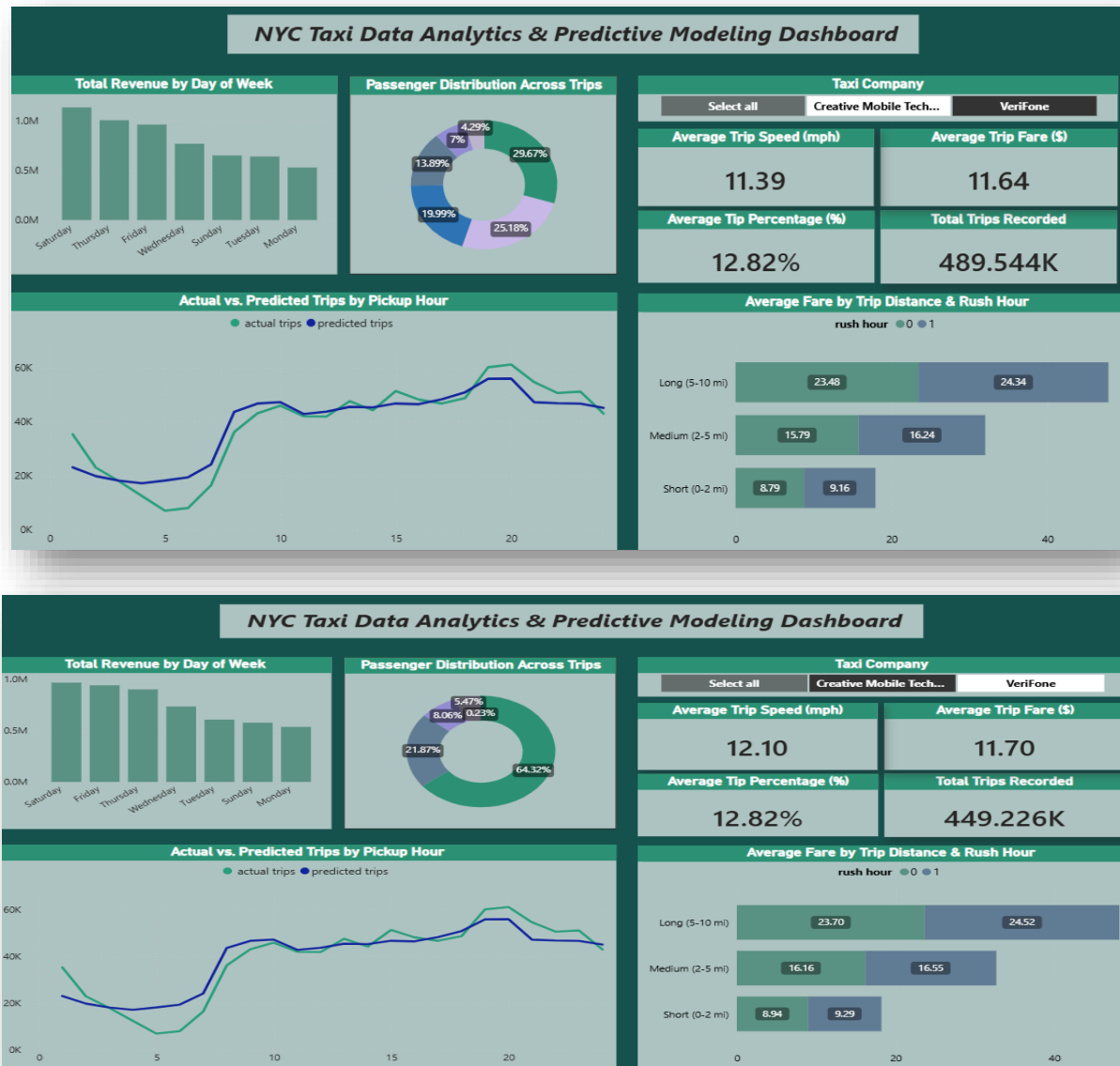


Figure 7 Comparative market and operational analysis between CMT and VeriFone vendors.

1. Market Share & Trip Volume

VeriFone leads with approximately 489.54K trips, representing the largest share of the dataset.

Creative Mobile Technologies (CMT) follows with 449.23K trips (47.8%), indicating a highly competitive market distribution.

2. Passenger Dynamics & Behavior

CMT (Focused Strategy): Heavily relies on single-passenger trips, accounting for 64.32% of total rides, indicating strong preference among solo commuters and business travelers.

VeriFone (Diverse Mobility): Exhibits a more balanced distribution across group sizes (2–5 passengers), representing over 45% of its trips, likely due to a more diverse vehicle fleet including larger cars such as SUVs and vans.

3. Operational Efficiency (Speed & Congestion)

CMT demonstrates higher efficiency with an average speed of 12.10 mph, suggesting better performance in less congested areas or more optimized routing.

VeriFone records a lower average speed of 11.39 mph, indicating operations in denser and more congested urban environments.

4. Financial Consistency

Both vendors show a nearly identical average tip percentage of 12.82%, indicating that tipping behavior is consistent across the market regardless of provider.

While both companies peak on Saturdays, VeriFone's ability to transport more passengers per trip provides a more stable revenue stream during high-demand weekend periods.

Key Strategic Insights & Recommendations

Based on the analytical findings, the following key insights and recommendations were derived:

Revenue Optimization: Since Saturday represents the highest revenue day (exceeding \$2M+ overall), fleet availability should be maximized during weekends.

Dynamic Pricing & Rush Hours: Fare analysis shows that Rush Hour 1 consistently generates higher fares across all trip distances (short, medium, and long), supporting the implementation of dynamic pricing strategies during congestion periods.

Operational Efficiency: The average trip speed of 11.73 mph reflects significant urban congestion. This highlights the need for improved traffic management and more efficient routing strategies.

Targeted Marketing: With over 40% of users being single passengers, loyalty programs and subscription models should primarily target individual commuters rather than group travelers.

Discussion

The project faced several technical and practical challenges during the development of the New York taxi data analysis system, mainly due to the large-scale and complex nature of the dataset.

One of the primary challenges was the high data volume, as the dataset contained millions of trip records collected over different time periods, with a total size of approximately 7 GB. Handling such a large dataset required significant computational resources and efficient processing techniques, making the use of Big Data technologies essential.

The dataset also included a considerable number of outliers in variables such as trip distance and fare amount. These abnormal values negatively affected the reliability and accuracy of the analysis and predictive modeling processes. Therefore, strict filtering and validation techniques were implemented to eliminate unrealistic records and improve the overall quality of the results.

From an infrastructure perspective, the project experienced performance limitations because the experiments were conducted in a constrained single-node environment using Google Colab. Although PySpark was utilized to simulate distributed processing logic, the environment itself did not provide a true distributed cluster architecture. Consequently, Google Drive was used as a practical cloud-based storage layer to mimic distributed data storage, serving as an alternative to enterprise-level solutions such as HDFS or Amazon S3. Furthermore, because the system was not deployed on a real multi-node cluster, the project could not fully represent industrial-scale distributed computing performance. Despite these limitations, the overall system architecture was designed with scalability and extensibility in mind.

In future work, the platform can be migrated to real cloud computing environments such as AWS EMR or Databricks, enabling true distributed processing, real-time streaming analytics, and significantly improved scalability and performance when handling much larger datasets.

Big Data Justification

This project is classified as a Big Data solution rather than a traditional data analysis task for several key reasons:

- **High Volume:** The project is based on a very large dataset containing millions of taxi trip records, which exceeds the processing capabilities of traditional tools such as Excel or Pandas.
- **Data Variety:** The dataset exhibits significant variety, as it includes temporal, spatial, financial, and behavioral attributes. This requires advanced techniques to extract hidden patterns and meaningful insights.
- **Distributed Processing:** The system relies on distributed processing, implemented using Apache Spark. This enables data processing across distributed memory rather than a single machine, which is a fundamental characteristic of Big Data systems.
- **Multi-Stage Pipeline:** The project is designed as a multi-stage data pipeline, including ingestion, cleaning, transformation, storage, machine learning, and visualization. This architecture reflects standard practices in modern Big Data ecosystems.
- **Real-World Scalability:** The system demonstrates high scalability. As taxi data is continuously generated in real time, the solution is designed to be capable of handling future data growth and potential streaming data sources.

Ethics & Privacy

This project fully adheres to data ethics and privacy principles. All datasets used are publicly available open data provided by the NYC Taxi and Limousine Commission (TLC), and do not contain any directly identifiable personal information.

Care was taken to ensure that no data capable of identifying passengers or drivers was used. The analysis was conducted at the trip level rather than the individual level.

In addition, the project follows the principle of responsible data usage, aiming to improve smart transportation systems and support decision-making without violating privacy or misusing data.

AI Tools Usage Declaration

Task	AI Tool Used	How It Was Used	Student Contribution
Debugging PySpark Code & Memory Issues	ChatGPT / Gemini	Suggested techniques to handle Out-Of-Memory (OOM) errors in Google Colab and provided syntax corrections for PySpark transformations.	We implemented the suggested solutions (e.g., taking a 5% sample for the ML pipeline), tested the code iteratively, and verified the final model's execution and accuracy.
Report Structuring	Gemini	Helped organize the report layout according to the assignment rubric, improved grammar, and refined technical phrasing for architectural concepts.	We established the core logic, raw data, and system architecture. Afterward, we audited and refined the AI's output to verify that it accurately and exclusively represented our original hands-on work.

Conclusion

This project presents an end-to-end Big Data system for analyzing urban transportation data in New York City, covering the full pipeline from large-scale data processing to predictive analytics and interactive visualization.

The results revealed clear patterns in taxi demand, particularly during evening peak hours, highlighting the strong impact of time and congestion on urban mobility. The machine learning model demonstrated good predictive performance in estimating demand and fare behavior based on historical data.

Furthermore, the analysis provided valuable insights into operational efficiency and the impact of traffic congestion on trip speed, supporting more informed transportation management decisions.

Overall, this project demonstrates the practical value of Big Data technologies in supporting smart cities, improving transportation efficiency, and enabling data-driven decision-making, with potential future extensions into real-time analytics and cloud-based scalable systems.

Resource

Power BI Dashboard (Interactive Visualization) :

<https://app.powerbi.com/view?r=eyJrIjoiZGY0YmJiOWYtMTA2OC00MjYwLWJiZWEtYWI4N2IxMDkyMTJmIiwidCI6Ijc5YTA1N2ZiLWIwZDUtNDRkZC04ZjkwLTBiZjcxNTFmNWMzZiIsImMiOiI9>

NYC Yellow Taxi Trip Dataset (Kaggle Source) :

<https://www.kaggle.com/datasets/elemento/nyc-yellow-taxi-trip-data?resource=download>

Google Colab Implementation (PySpark Big Data Pipeline Notebook):

https://colab.research.google.com/drive/1J_MRkLdDzyctU9khc0eDnu1ePS4GsybN?usp=sharing